

MCS-012 Assignment Solutions Guide

Question 1: Computer Organization Fundamentals

(a) Instruction Execution Simulation

(i) Initial State Diagram

Memory Contents (Hexadecimal):

- Location 10F: LOAD A instruction
- Location 110: SUBT B instruction
- Location 111: STOR C instruction
- Location 2FF: 2AC5 (Operand A)
- Location 300: 1AEE (Operand B)
- Location 301: 0000 (Operand C)

Register Initial Values:

- PC: 010E (12 bits)
- AC: 0000 (16 bits)
- IR: 0000 (16 bits)
- MAR: 0000 (16 bits)
- MBR: 0000 (16 bits)

Instruction Encoding:

- LOAD opcode: 0001, SUBT opcode: 0010, STOR opcode: 0011
- LOAD A = 0001 0010 1111 1111 (12FF in hex)
- SUBT B = 0010 0011 0000 0000 (2300 in hex)
- STOR C = 0011 0011 0000 0001 (3301 in hex)

(ii) State After Each Instruction

After LOAD A:

- PC: 010F
- AC: 2AC5
- IR: 12FF
- Memory unchanged

After SUBT B:

- PC: 0110
- AC: 0FD7 (2AC5 - 1AEE = 0FD7)
- IR: 2300
- Memory unchanged

After STOR C:

- PC: 0111
- AC: 0FD7 (unchanged)
- IR: 3301
- Memory 301: 0FD7

(b) Number System Conversions

i) Decimal (269785421)₁₀ to Binary and Hexadecimal

Binary: 10000000101100011000011001101₂

Hexadecimal: 1016186D₁₆

Method: Repeatedly divide by 2 for binary, group by 4 bits for hex.

ii) Hexadecimal (8FACBED34)_h to Octal

Steps:

1. Convert hex to binary: 1000 1111 1010 1100 1011 1110 1101 0011 0100
2. Group in 3s from right: 010 001 111 110 101 100 101 111 101 101 000 110 100
3. **Octal:** 21776547556064₈

iii) String "lowercase vs UPPERCASE" to UTF-8

l: 6C, o: 6F, w: 77, e: 65, r: 72, c: 63, a: 61, s: 73, e: 65
(space): 20
v: 76, s: 73
(space): 20
U: 55, P: 50, P: 50, E: 45, R: 52, C: 43, A: 41, S: 53, E: 45

Complete: 6C 6F 77 65 72 63 61 73 65 20 76 73 20 55 50 50 45 52 43 41 53 45

iv) Octal (65874531)₈ to Decimal

Steps:

1. $(6 \times 8^7) + (5 \times 8^6) + (8 \times 8^5)$ - INVALID! Octal digits are 0-7 only.
2. Assuming valid octal (65074531)₈:
3. **Decimal:** 13954393₁₀

(c) K-map Simplification

Function: $F(A,B,C,D) = \Sigma(0,2,6,8,9,12)$

K-map Layout:

			CD	
AB	00	01	11	10

00		1	0	0	1	(0, 2)
01		0	0	0	0	
10		1	1	0	0	(8, 9)
11		1	0	0	0	(12)

Simplified Expression: $F = A'B'D' + A'BC' + AB'C'D'$

NAND Implementation: Use De Morgan's theorem to convert AND-OR to NAND-NAND.

(d) Adder-Subtractor Circuit (A=0011, B=1111, Operation: A-B)

For subtraction, B is complemented and 1 is added (2's complement).

Inputs:

- A = 0011 (A3A2A1A0)
- B = 1111, complemented = 0000 (when XORed with M=1)
- M (mode) = 1, Cin to FA0 = 1

Computation:

- B' = 0000, Adding 1: 0001
- 0011 + 0000 + 1 = 0100

Outputs:

- S3S2S1S0 = 0100 (result = 4, which is -12 in 2's complement due to overflow)
- Carry out = 0
- **Overflow:** Yes (positive + positive = negative in 4-bit)

(e) 4×1 Multiplexer

Function: Selects one of 4 inputs based on 2 select lines.

Inputs: I0, I1, I2, I3, S0, S1 **Output:** Y

Truth Table:

- S1 S0 = 00 → Y = I0
- S1 S0 = 01 → Y = I1
- S1 S0 = 10 → Y = I2
- S1 S0 = 11 → Y = I3

Example: If I0=0, I1=1, I2=0, I3=1 and S1S0=01, then Y=1

(f) Hamming Code Error Correction

Source Data: 1001 (D4 D3 D2 D1) **Received Data:** 1000

Hamming Code Construction:

- Parity bits at positions 1, 2, 4 (P1, P2, P4)
- Data bits at positions 3, 5, 6, 7

7-bit code: P1 P2 D1 P4 D2 D3 D4 Position: 1 2 3 4 5 6 7

Original (1001):

- Position 3=1, 5=0, 6=0, 7=1
- P1 (checks 1,3,5,7): $1 \oplus 0 \oplus 0 \oplus 1 = 0$
- P2 (checks 2,3,6,7): $1 \oplus 0 \oplus 0 \oplus 1 = 0$
- P4 (checks 4,5,6,7): $0 \oplus 0 \oplus 0 \oplus 1 = 1$
- **Code:** 0011001

Received (1000):

- Code would be: 0011000
- Check: P1=0, P2=0, P4=1
- Error at position: $4+2+1 = 7 \checkmark$
- Correct bit 7: Changes to 1001

(g) JK Flip-Flop

Characteristic Table:

J K Q(t+1) Description

0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Toggle

Excitation Table:

Q(t) Q(t+1) J K

0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

(h) Master-Slave Flip-Flop

Structure: Two flip-flops in series with inverted clock.

Operation:

1. When CLK=1: Master is enabled, slave is disabled
2. Master captures input
3. When CLK=0: Master disabled, slave enabled

4. Slave copies master's state to output

Advantage: Eliminates race conditions and timing issues.

(i) IEEE 754 Representation

$(321.25)_{10}$

Binary: $101000001.01_2 = 1.0100000101 \times 2^8$

Single Precision (32-bit):

- Sign: 0
- Exponent: $8 + 127 = 135 = 10000111$
- Mantissa: 010000010100000000000000
- **Result:** 0 10000111 010000010100000000000000

Double Precision (64-bit):

- Sign: 0
- Exponent: $8 + 1023 = 1031 = 10000000111$
- Mantissa: 0100000101000...
- **Result:** 0 10000000111 010000010100

$(-7.125)_{10}$

Binary: $-111.001_2 = -1.11001 \times 2^2$

Single Precision:

- Sign: 1
- Exponent: $2 + 127 = 129 = 10000001$
- Mantissa: 110010000000000000000000
- **Result:** 1 10000001 110010000000000000000000

Question 2: Memory and I/O Systems

(a) 8×4 RAM Organization

Structure:

- 8 words × 4 bits per word
- Total: 32 bits of storage

(i) Data Lines:

- **Data Input:** 4 lines (Din0-Din3)
- **Data Output:** 4 lines (Dout0-Dout3)
- Each word is 4 bits wide

(ii) Address Lines:

- **3 address lines** (A0, A1, A2)
- Reason: $2^3 = 8$ addresses needed for 8 words

Other Signals:

- Read/Write control
- Chip Select (CS)
- Output Enable (OE)

(b) Cache Mapping for Address (1A9)_h

Given:

- Main Memory: 4K words = 4096 words = 2^{12} words
- Word size: 16 bits
- Cache: 8 blocks $\times$ 32 bits (2 words per block)
- Address 1A9_h = 0001 1010 1001₂ (12 bits)

(i) Direct Mapping

- Block address = $1A9/2 = D4$ ($\div 2$ as 2 words/block)
- Cache block = $D4 \bmod 8 = D4 \bmod 8 = 4$
- **Tag:** 1A (bits 11-3)
- **Index:** 4 (bits 2-0)
- **Offset:** 1 bit (word within block)

(ii) Associative Mapping

- **Tag:** 1A9 (entire address except offset)
- **Index:** None (can go to any block)
- **Offset:** 1 bit

(iii) Two-way Set Associative

- $8 \text{ blocks} \div 2 = 4 \text{ sets}$
- Set number = $(D4 \bmod 4) = 0$
- **Tag:** bits for identifying within set
- **Set Index:** 00 (2 bits for 4 sets)
- **Offset:** 1 bit

(c) Interrupts

Definition: An interrupt is a signal that temporarily stops the CPU from executing the current program to handle an urgent event.

Uses:

- Handle I/O operations without busy waiting
- Handle hardware errors
- Implement multitasking
- Handle timer events

Interrupt Handling Process:

1. Device sends interrupt signal
2. CPU completes current instruction
3. CPU saves Program Counter and status
4. CPU disables further interrupts
5. CPU jumps to Interrupt Service Routine (ISR)
6. ISR executes
7. CPU restores saved state
8. CPU re-enables interrupts
9. CPU resumes normal execution

(d) I/O Processor

Definition: An I/O processor (channel) is a specialized processor that handles I/O operations independently of the CPU.

Uses:

- Offload I/O operations from CPU
- Handle multiple devices simultaneously
- Improve system throughput
- Execute I/O programs

Structure:

- Control unit
- ALU for address calculations
- Registers for I/O commands
- Interface to devices
- Interface to main memory
- DMA capability

(e) File Allocation on Disk**Given:**

- 64 tracks $\times$ 32 sectors $\times$ 4MB per sector
- Cluster size: 4 sectors = 16 MB
- File size: 32 MB
- First 4 clusters occupied by OS

Solution:

- File needs: $32 \text{ MB} \div 16 \text{ MB} = 2$ clusters
- Available: Clusters 5 onwards
- Allocate: Clusters 5 and 6

FAT Entries:

Cluster Next Cluster

0-3	OS (marked)
4	Free
5	6
6	EOF
7+	Free

(f) Brief Explanations

(i) Latency Time: Rotational delay for desired sector to reach read/write head. Average is half rotation time. Affects access speed.

(ii) Non-impact Printers: Printers without physical contact with paper (inkjet, laser). Advantages: Quiet, fast, high quality. Disadvantages: Expensive consumables.

(iii) MODEM: Modulator-Demodulator converts digital signals to analog for transmission over telephone lines and vice versa. Essential for internet over phone lines.

(iv) Video Memory: Dedicated RAM for storing frame buffer and graphics data. Faster than system RAM for graphics operations. Size determines maximum resolution/color depth.

(v) Colour Depth: Bits per pixel determining color range. 8-bit = 256 colors, 24-bit = 16.7M colors. Higher depth = better quality, more memory needed.

(vi) Magnetic Tape: Sequential storage medium. Advantages: High capacity, low cost, reliable for backup. Disadvantages: Slow sequential access, not for random access.

Question 3: Instruction Set and Control

(a) Instruction Format Design

(i) Instruction Format:

Requirements:

- 32-bit instruction
- 32 opcodes $\rightarrow$ 5 bits
- 32 registers $\rightarrow$ 5 bits
- $64 \text{ KB} = 2^{16}$ words $\rightarrow$ 16 bits for memory address

Format:

| Opcode (5) | Register (5) | Memory Address (16) | Unused (6) |

Register Sizes:

- General Purpose: 32 bits each (32 registers)
- PC: 16 bits (to address 64K words)
- MAR: 16 bits
- DR: 32 bits
- IR: 32 bits
- FR: 16 bits (flag register)

Unused bits: 6 bits (could be used for addressing modes)

(ii) Valid Instructions:

Example 1: ADD R5, [2000]

Opcode: 00001 (ADD)
Register: 00101 (R5)
Address: 0000100000000000 (2000h)

Example 2: SUB R10, [3FFF]

Opcode: 00010 (SUB)
Register: 01010 (R10)
Address: 0011111111111111 (3FFFh)

(iii) Subtraction Instruction Micro-operations:

Instruction: SUB R2, [ADDR] (Accumulator = R2 - M[ADDR])

Fetch Cycle:

1. MAR $\leftarrow$ PC
2. MBR $\leftarrow$ M[MAR], PC $\leftarrow$ PC + 1
3. IR $\leftarrow$ MBR

Decode & Execute: 4. MAR $\leftarrow$ IR(address field) 5. DR $\leftarrow$ M[MAR] (fetch operand) 6. AC $\leftarrow$ R2 (move register to accumulator) 7. AC $\leftarrow$ AC - DR (perform subtraction) 8. FR $\leftarrow$ set flags based on result

(b) Micro-operations Results

Initial: R1 = 10010001, R2 = 00100100

(i) Decrement R2:

- Select: S1S0 = 10, Cin = 0
- Result: 00100011

- Carry-out: 0

(ii) Add R1 and R2:

- Select: S1S0 = 10, Cin = 0 (with both inputs)
- Result: 10110101
- Carry-out: 0

(iii) OR R1 and R2:

- Select: S1S0 = 01
- Result: 10110101
- Carry-out: 0

(iv) Shift Right R1:

- Select: S1S0 = 00 (with shift)
- Result: 01001000
- Carry-out: 1 (LSB shifted out)

(c) Instruction Pipeline

Pipeline Diagram:

Clock	FED	OFX	STOR
-----	-----	-----	-----
1	I1		
2	I2	I1	
3	I3	I2	I1
4	I4	I3	I2
5	I5	I4	I3
6		I5	I4
7			I5

Problem with Conditional Jump (I1): If I1 is a conditional jump, the pipeline doesn't know if the jump will be taken until after OFX stage. Instructions I2 and I3 may have already been fetched but might need to be discarded (pipeline flush). This causes pipeline stalls and reduces efficiency.

Solutions: Branch prediction, delayed branching

(d) Wilkes Control Unit

Structure:

- Uses a diode matrix (control memory)
- Each row represents a micro-instruction
- Columns represent control signals
- Conditional logic for sequencing

Operation:

1. Instruction decoder selects starting row
2. Row activates control signals via diodes
3. Sequencer determines next micro-instruction
4. Process continues until instruction complete

Comparison with Microprogrammed Control:

Similarities:

- Both use sequences of micro-operations
- Both provide flexible control

Differences:

- Wilkes uses hardware matrix (faster, fixed)
- Microprogrammed uses writable control memory (flexible, slower)
- Wilkes is cheaper for simple processors
- Microprogrammed better for complex CISC

(e) RISC Characteristics

Key Characteristics:

1. Simple, fixed-length instructions
2. Load/store architecture
3. Large number of registers (32+)
4. Hardwired control (no microcode)
5. Single-cycle execution
6. Simple addressing modes
7. Pipelining friendly

Uses of Large Register Set:

- Reduce memory access (faster)
- Hold frequently used variables
- Support register windowing
- Enable efficient parameter passing
- Reduce load/store traffic
- Improve compiler optimization
- Support deep pipelining

Question 4: 8086 Assembly Programming

(a) Assembly Program

```
; Program to generate sequence in array
; Input: One digit from keyboard
; Output: BL values stored in 10-byte array
```

```

.MODEL SMALL
.STACK 100H

.DATA
    ARRAY DB 10 DUP(?)      ; Array of 10 bytes
    PROMPT DB 'Enter a digit (0-9): $'

.CODE
MAIN PROC
    ; Initialize data segment
    MOV AX, @DATA
    MOV DS, AX

    ; Display prompt
    LEA DX, PROMPT
    MOV AH, 09H              ; DOS function to display string
    INT 21H

    ; Read single character from keyboard
    MOV AH, 01H              ; DOS function to read character
    INT 21H                  ; AL now contains ASCII digit

    ; Convert ASCII to binary
    SUB AL, 30H              ; Convert ASCII to numeric value
    MOV BL, AL               ; Store in BL

    ; Initialize loop counter and array pointer
    MOV CX, 10               ; Loop 10 times
    LEA SI, ARRAY            ; SI points to array start

STORE_LOOP:
    ; Store BL value in current array location
    MOV [SI], BL             ; Store BL at address pointed by SI

    ; Increment BL for next iteration
    INC BL                   ; BL = BL + 1

    ; Move to next array location
    INC SI                   ; Point to next byte

    ; Continue loop
    LOOP STORE_LOOP          ; Decrement CX and jump if CX != 0

    ; Exit program
    MOV AH, 4CH              ; DOS function to exit
    INT 21H

MAIN ENDP
END MAIN

```

(b) FAR vs NEAR Procedure Calls

NEAR Procedure:

- Within same code segment
- Only IP (Instruction Pointer) saved on stack
- 2 bytes pushed
- Return address is offset only

- RET instruction pops IP

FAR Procedure:

- Can be in different code segment
- Both CS and IP saved on stack
- 4 bytes pushed (CS first, then IP)
- Return address includes segment and offset
- RET FAR pops both IP and CS

Parameter Passing (3-word parameters):

```
Stack Layout (growing downward):
+-----+ ← SP (after call)
| IP (2 bytes) |
+-----+
| CS (2 bytes) |
+-----+
| Param 3 (2 bytes) |
+-----+
| Param 2 (2 bytes) |
+-----+
| Param 1 (2 bytes) | ← BP+10
+-----+
```

In Procedure:

```
PROC_NAME PROC FAR
    PUSH BP                ; Save old BP
    MOV BP, SP             ; Set up stack frame

    ; Access parameters
    MOV AX, [BP+10]        ; Param 1
    MOV BX, [BP+12]        ; Param 2
    MOV CX, [BP+14]        ; Param 3

    ; ... procedure code ...

    POP BP                ; Restore BP
    RET 6                  ; Return and clean 6 bytes (3 params)
PROC_NAME ENDP
```

(c) 8086 Concepts

(i) CS:IP and SS:SP Addressing

CS:IP (Code Segment:Instruction Pointer):

- Physical Address = CS × 16 + IP
- Example: If CS = 2000H, IP = 0500H
- Physical = 20000H + 0500H = 20500H
- Points to next instruction to execute

SS:SP (Stack Segment:Stack Pointer):

- Physical Address = $SS \times 16 + SP$
- Example: If $SS = 3000H$, $SP = 0100H$
- Physical = $3000H \times 16 + 0100H = 30100H$
- Points to top of stack
- SP decrements on PUSH, increments on POP

(ii) Flags

CF (Carry Flag):

- Set when unsigned arithmetic produces carry/borrow
- Used in multi-precision arithmetic
- Example: ADD AL, BL (if result > 255, CF = 1)

SF (Sign Flag):

- Copy of MSB of result
- Indicates positive (0) or negative (1) for signed numbers
- Example: After SUB, if result is negative, SF = 1

PF (Parity Flag):

- Set if result has even number of 1s in lower byte
- Used for error detection
- PF = 1 if even parity, 0 if odd

IF (Interrupt Flag):

- Enables (1) or disables (0) maskable interrupts
- CLI clears it, STI sets it
- Critical sections use CLI to prevent interruption

(iii) Instructions

LEA (Load Effective Address):

```
LEA BX, [SI+4] ; BX = address of SI+4 (not content)
; Used to get pointer/address without accessing memory
```

CMP (Compare):

```
CMP AX, BX ; Performs AX - BX, sets flags but doesn't store result
; Used before conditional jumps
; Example: CMP AX, 10
; JE EQUAL ; Jump if AX = 10
```

SHR (Shift Logical Right):

```
SHR AL, 1 ; Shift AL right by 1 bit, 0 enters from left
; Before: AL = 10110101
; After: AL = 01011010, CF = 1 (shifted out bit)
; Divides by 2 for unsigned numbers
```

RCR (Rotate Right through Carry):

```
RCR AL, 1          ; Rotate AL right through CF
; Before: AL = 10110101, CF = 1
; After:  AL = 11011010, CF = 1
; CF becomes MSB, LSB goes to CF
; Used for multi-precision shifts
```
